Kubernetes Debug Pods & Java Debugging Cheat Sheet

Recommended images, capabilities, commands to run inside them, and the most useful kubectl launch / attach patterns

Scope: Kubernetes/k3s environments running Java and Spring Boot workloads. The emphasis is on a small, high-value toolbox instead of dozens of overlapping images.

Recommended toolbox

Purpose	Recommended image	Best use
Network / DNS / TLS / packets	nicolaka/netshoot	Connectivity, routing, packet capture, HTTP/TLS, gRPC, bandwidth
JVM diagnostics	eclipse-temurin:21-jdk-jammy	jcmd, JFR, thread dumps, heap dumps, GC and JVM inspection
Live Java / Spring debugging	hengyunabc/arthas	Trace methods, inspect args/results, decompile classes, live profiling
Database debugging	tazhate/k8s-debug-container-db	PostgreSQL, MySQL, Redis, MongoDB and SQLite client tests
Storage / PVC debugging	tazhate/k8s-debug-container-storage	fio, ioping, disk/PVC latency and S3-compatible storage
Linux / performance debugging	tazhate/k8s-debug-container-perf	strace, perf, htop, BCC tools and process analysis
Deep node-wide profiling	intel/gprofiler	Java + native + kernel CPU profiling and flame graphs
Continuous Java profiling	grafana/alloy + Pyroscope	Historical CPU, allocation, wall-clock and lock profiles

Operational rule: use the least-privileged mode that answers the question. Start with logs/describe/top, then escalate to ephemeral containers, specialist Pods, and finally node-level privileged debugging.

Common variables used in examples

```
NS=taskflow
POD=taskflow-backend-xxxxxxxxxx-xxxxx
CONTAINER=taskflow-backend

NODE=$(kubectl get pod -n "$NS" "$POD" \
  -o jsonpath='{.spec.nodeName}')
```

1. Netshoot - network, DNS, TLS and packet debugging

Image: `nicolaka/netshoot`

Best for

- DNS and service discovery
- TCP/UDP connectivity
- HTTP/HTTPS and TLS
- Routes, interfaces and sockets
- Packet capture
- Latency, packet loss and bandwidth
- Port scanning
- gRPC and WebSocket testing
- iptables/nftables and conntrack investigation

Recommended launch - same application Pod

```
kubectl debug -n "$NS" -it "pod/$POD" \
  --container="netshoot-$(date +%s)" \
  --target="$CONTAINER" \
  --image=nicolaka/netshoot \
  --profile=netadmin \
  -- zsh
```

Use profile=general for simple DNS/HTTP checks. Use profile=netadmin when you need packet capture, firewall/routing inspection, or deeper networking.

Commands inside Netshoot

DNS

```
cat /etc/resolv.conf
dig taskflow-backend.taskflow.svc.cluster.local
drill taskflow-backend.taskflow.svc.cluster.local
nslookup taskflow-backend
```

HTTP / HTTPS

```
curl -v http://taskflow-backend:8080/actuator/health
curl -I https://example.com
curl -vk https://example.com
```

TLS

```
openssl s_client -connect example.com:443 -servername example.com
```

TCP connectivity

```
nc -vz taskflow-backend 8080
```

Routes and interfaces

```
ip addr
ip route
ip neigh
```

Sockets

```
ss -tulpn
ss -tan
ss -s
```

Packet capture

```
tcpdump -i any -nn
tcpdump -i any -nn port 8080
tcpdump -i any -nn -s0 -w /tmp/incident.pcap
```

Latency / routing

```
mtr -rw taskflow-backend
```

Port scan

```
nmap taskflow-backend
```

Bandwidth

```
iperf3 -c <IP>
```

gRPC

```
grpcurl <service>:9090 list
```

WebSocket

```
websocat ws://service:8080/ws
```

2. Eclipse Temurin JDK - general JVM diagnostics

Image: `eclipse-temurin:21-jdk-jammy`

Best for

- Thread dumps and deadlocks
- Heap dumps and class histograms
- Garbage-collection inspection
- JVM flags and system properties
- Native Memory Tracking
- Java Flight Recorder (JFR)

- Class loading and JVM configuration

Recommended launch

```
kubectl debug -n "$NS" -it "pod/$POD" \
  --container="java-debug-$(date +%s)" \
  --target="$CONTAINER" \
  --image=eclipse-temurin:21-jdk-jammy \
  --profile=general \
  -- bash
```

Prefer a debug JDK with the same Java major version as the target application. If JVM attach is blocked by permissions, retry with `--profile=sysadmin` only when necessary.

Commands inside the JDK debug container

Find Java processes

```
ps -ef
jcmd -l
```

JVM details

```
jcmd 1 VM.version
jcmd 1 VM.flags
jcmd 1 VM.system_properties
```

Thread dump / deadlock investigation

```
jcmd 1 Thread.print
jcmd 1 Thread.print -l
```

Heap histogram

```
jcmd 1 GC.class_histogram
```

Heap dump

```
jcmd 1 GC.heap_dump /tmp/heap.hprof
```

GC statistics

```
jstat -gc 1 1000
```

Native memory

```
jcmd 1 VM.native_memory summary
```

60-second JFR recording

```
jcmd 1 JFR.start name=incident settings=profile duration=60s filename=/tmp/incident.jfr
jcmd 1 JFR.check
```

Native Memory Tracking must already be enabled in the target JVM for full NMT detail.

3. Arthas - live Java and Spring Boot debugging

Image: `hengyunabc/arthas`

Best for

- Live method tracing
- Inspecting arguments, return values and exceptions
- Hot or blocked threads
- Decompiling classes actually loaded in production
- Classloader problems
- Logging inspection
- Async-profiler CPU profiling
- Spring service-level debugging

Recommended launch

```
kubectl debug -n "$NS" -it "pod/$POD" \
  --container="arthas-$(date +%s)" \
  --target="$CONTAINER" \
  --image=hengyunabc/arthas \
  --profile=sysadmin \
  -- /bin/sh

java -jar /opt/arthas/arthas-boot.jar
```

Commands inside Arthas

Overview

```
dashboard
```

Busy threads

```
thread -n 10
```

Blocked threads

```
thread --state BLOCKED
```

JVM and memory

```
jvm
memory
```

Loaded classes

```
sc com.example.*
```

Methods

```
sm com.example.service.OrderService
```

Decompile the actual runtime class

```
jad com.example.service.OrderService
```

Trace a method

```
trace com.example.service.OrderService createOrder
```

Inspect args / return / exception

```
watch com.example.service.OrderService createOrder '{params,returnObj,throwExp}' -x 2
```

Show callers

```
stack com.example.service.OrderService createOrder
```

Profile CPU

```
profiler start
# reproduce workload
profiler stop
```

Stop Arthas

```
stop
```

Arthas instruments live classes. Keep trace/watch patterns narrow, use short sessions, and stop instrumentation after the incident.

4. Database debug Pod

Image: `tazhate/k8s-debug-container-db`

Best for

- PostgreSQL
- MySQL
- Redis
- MongoDB
- SQLite
- Database authentication and application-like connectivity tests from a namespace

Recommended launch - separate Pod

```
kubectl run db-debug \
  -n "$NS" \
  --rm -it \
  --restart=Never \
  --image=tazhate/k8s-debug-container-db \
  -- bash
```

A separate Pod is usually better here because it tests connectivity as another workload in the namespace would experience it.

Commands inside

PostgreSQL

```
psql -h postgres -p 5432 -U taskflow -d taskflow
```

PostgreSQL readiness

```
pg_isready -h postgres -p 5432
```

MySQL

```
mysql -h mysql -u taskflow -p
```

Redis

```
redis-cli -h redis ping
```

MongoDB

```
mongosh mongodb://mongodb:27017
```

5. Storage / PVC debug Pod

Image: `tazhate/k8s-debug-container-storage`

Best for

- PVC latency and throughput
- Filesystem capacity
- Large-directory investigation
- fio and ioping
- S3-compatible storage clients
- Disk and I/O analysis

PVC-mount example

```
apiVersion: v1
kind: Pod
metadata:
  name: storage-debug
  namespace: taskflow
spec:
  restartPolicy: Never
  containers:
    - name: debug
      image: tazhate/k8s-debug-container-storage
      command: ["sleep", "infinity"]
      volumeMounts:
        - name: data
          mountPath: /data
  volumes:
    - name: data
      persistentVolumeClaim:
        claimName: YOUR-PVC
```

```
kubectl apply -f storage-debug.yaml
kubectl exec -n taskflow -it storage-debug -- bash
```

Commands inside

Capacity / directory usage

```
df -h
du -sh /data/*
ncdu /data
```

Latency

```
ioping -c 20 /data
```

Sequential read

```
fio --name=read --directory=/data --rw=read --bs=1M --size=1G --runtime=30
```

Random 4K read

```
fio --name=randread --directory=/data --rw=randread --bs=4k --size=1G --runtime=60
```

Warning: fio write workloads can modify or destroy data. Use a dedicated test file/directory and never point destructive tests at live application data.

6. Performance / Linux debug Pod

Image: `tazhate/k8s-debug-container-perf`

Best for

- Linux process debugging
- System-call tracing
- CPU profiling
- Disk activity
- Scheduler behaviour
- Process hangs
- BCC/eBPF tools where the node kernel permits them

Recommended launch

```
kubectl debug -n "$NS" -it "pod/$POD" \
  --container="perf-debug-$(date +%s)" \
  --target="$CONTAINER" \
  --image=tazhate/k8s-debug-container-perf \
  --profile=sysadmin \
  -- bash
```

Commands inside

Processes

```
ps aux
htop
```

Trace syscalls

```
strace -p 1
```

Trace networking syscalls

```
strace -f -e trace=network -p 1
```

Open file descriptors

```
ls -l /proc/1/fd
```

Live CPU profile

```
perf top
```

Record CPU profile

```
perf record -p 1 -g -- sleep 30
perf report
```

perf/BCC behaviour depends on node kernel settings and privileges. Use sysadmin only when a deep host/process investigation justifies it.

7. gProfiler - deep node-wide CPU profiling

Image: `intel/gprofiler`

Best for

- Whole-node CPU profiling
- Java and native code together
- JVM internals
- Kernel stacks
- Multiple processes
- Flame graphs for difficult CPU incidents

Recommended launch

```
NODE=$(kubectl get pod -n "$NS" "$POD" \
  -o jsonpath='{.spec.nodeName}')

kubectl debug -it "node/$NODE" \
  --container=gprofiler \
  --image=intel/gprofiler \
  --profile=sysadmin
```

Treat gProfiler as a privileged, one-off profiler. For ordinary Java CPU problems, start with JFR or Arthas. Pin the exact gProfiler image version and document the profiling flags you validate internally.

8. Grafana Alloy + Pyroscope - continuous profiling

Image: `grafana/alloy`

Best for

- Continuous Java CPU profiles
- Wall-clock profiles
- Allocation profiles
- Lock contention
- Historical flame graphs
- Comparing behaviour before and after deployments

Deploy as a DaemonSet, not as an ad-hoc debug shell

```
helm repo add grafana https://grafana.github.io/helm-charts
helm repo update

helm upgrade --install alloy-java grafana/alloy \
  --namespace monitoring \
  --create-namespace \
  --values alloy-java-values.yaml
```

Alloy/Pyroscope is persistent observability. Its value is that you can inspect what Java was doing before a restart or alert instead of having to reproduce the incident.

Core kubectl debugging patterns

A. exec - existing container already has a shell

Starts a new process inside the existing container.

```
kubectl exec -n "$NS" -it "$POD" -c "$CONTAINER" -- bash
# or
kubectl exec -n "$NS" -it "$POD" -c "$CONTAINER" -- sh
```

B. debug - add an ephemeral container to the existing Pod

Best for distroless/minimal images and same-Pod investigation.

```
kubectl debug -n "$NS" -it "pod/$POD" \
  --container="debug-$(date +%s)" \
  --target="$CONTAINER" \
  --image=<DEBUG-IMAGE> \
  --profile=general \
  -- bash
```

Examples: Netshoot -> profile=netadmin / zsh. Temurin -> profile=general / bash. Perf -> profile=sysadmin / bash.

C. run - completely separate throwaway debug Pod

Best for Service, DNS, NetworkPolicy, DB connectivity and cross-Pod testing.

```
kubectl run netshoot \
  -n "$NS" \
  --rm -it \
  --restart=Never \
  --image=nicolaka/netshoot \
  -- zsh
```

D. reusable temporary debug Pod

```
kubectl run netshoot \
  -n "$NS" \
  --restart=Never \
  --image=nicolaka/netshoot \
  --command -- sleep infinity

kubectl exec -n "$NS" -it netshoot -- zsh
kubectl delete pod -n "$NS" netshoot
```

E. debug a Kubernetes node

Useful for k3s, Cilium, containerd, host routing, host filesystem and node-level packet capture.

```
kubectl debug -it "node/$NODE" \
  --image=nicolaka/netshoot \
  --profile=sysadmin \
  -- zsh

# Host filesystem is available under /host
ls /host
chroot /host
```

For k3s, useful host paths commonly include /etc/rancher/k3s, /var/lib/rancher/k3s, /var/lib/kubelet and /run/k3s.

F. copy a Pod for experimentation

Useful when the app crashes immediately, startup is broken, or you want to experiment without modifying the original Pod.

```
kubectl debug -n "$NS" "$POD" -it \
  --copy-to="${POD}-debug" \
  --image=eclipse-temurin:21-jdk-jammy \
  --share-processes \
  -- bash

# Replace an image in the copied Pod
kubectl debug -n "$NS" "$POD" \
  --copy-to="${POD}-debug" \
  --set-image="$CONTAINER=eclipse-temurin:21-jdk-jammy"
```

G. attach vs exec

exec starts a new process. attach connects to a container's already-running main process. For troubleshooting, exec is normally more useful.

```
kubectl exec -it "$POD" -c netshoot -- zsh
kubectl attach -it "$POD" -c netshoot
```

H. re-enter a long-running debug container

```
kubectl exec -n "$NS" -it "$POD" -c netshoot -- zsh
```

Ephemeral containers are not restarted and cannot be removed independently from the Pod. Using unique container names such as netshoot-$(date +%s) avoids name collisions across sessions.

I. copy diagnostic artifacts out

```
# PCAP
kubectl cp -n "$NS" -c netshoot \
  "$POD:/tmp/incident.pcap" ./incident.pcap

# Heap dump
kubectl cp -n "$NS" -c java-debug \
  "$POD:/tmp/heap.hprof" ./heap.hprof

# JFR
kubectl cp -n "$NS" -c java-debug \
  "$POD:/tmp/incident.jfr" ./incident.jfr

# If tar is missing in the container
kubectl exec -n "$NS" "$POD" -c java-debug -- \
  cat /tmp/incident.jfr > incident.jfr
```

J. port-forward to a Pod, Deployment or Service

```
kubectl port-forward -n "$NS" pod/"$POD" 8080:8080
kubectl port-forward -n "$NS" deployment/taskflow-backend 8080:8080
kubectl port-forward -n "$NS" service/taskflow-backend 8080:8080
```

Check these before launching a debug image

Logs

```
kubectl logs -n "$NS" "$POD" -c "$CONTAINER"
kubectl logs -n "$NS" "$POD" -c "$CONTAINER" -f
kubectl logs -n "$NS" "$POD" -c "$CONTAINER" --previous
kubectl logs -n "$NS" "$POD" -c "$CONTAINER" --since=1h
kubectl logs -n "$NS" "$POD" --all-containers=true
```

`--previous` is especially valuable for CrashLoopBackOff because it shows logs from the previous terminated container.

Pod state

```
kubectl describe pod -n "$NS" "$POD"
kubectl get pod -n "$NS" "$POD" -o yaml
kubectl get pod -n "$NS" "$POD" -o wide
kubectl get pod -n "$NS" "$POD" -o jsonpath='{.spec.containers[*].name}{"\n"}'
kubectl get pod -n "$NS" "$POD" -o jsonpath='{.spec.nodeName}{"\n"}'
```

Resource usage

```
kubectl top pod -n "$NS" "$POD"
kubectl top pod -n "$NS" "$POD" --containers
kubectl top pods -n "$NS" --sort-by=cpu
kubectl top nodes
```

Which debug image should I use?

Symptom / question	First choice	Escalate to
Service cannot be reached	Netshoot	Node-level Netshoot if routing/Cilium looks wrong
DNS failure	Netshoot	Node-level Netshoot / Cilium investigation
TLS/certificate issue	Netshoot	Application/JVM truststore inspection
High Java CPU	Temurin JFR	Arthas; gProfiler if native/kernel involvement suspected
Java app hangs	Temurin thread dump	Arthas, perf/strace
Memory leak / OOM	Temurin histogram + heap dump	JFR / Pyroscope allocation profiling
Wrong Spring method behaviour	Arthas	JFR / application traces
Database login/connectivity	DB debug Pod	Netshoot for lower-level network/TLS
Slow PVC	Storage debug Pod	Node/storage backend investigation
Native/system-call problem	Performance debug Pod	gProfiler / node debugging
Historical Java CPU spike	Alloy + Pyroscope	JFR/gProfiler for focused reproduction
Cilium/k3s/containerd/node issue	Node-level Netshoot	Host tools / gProfiler depending on symptom

Recommended troubleshooting workflow

- 1. kubectl describe pod
- 2. kubectl logs, including --previous
- 3. kubectl top pod --containers
- 4. Netshoot for network / DNS / TLS
- 5. Temurin JDK for jcmd, thread dumps, heap dumps and JFR
- 6. Arthas for method-level application behaviour
- 7. perf / gProfiler when OS, native or kernel behaviour is involved
- 8. Node debug when the problem appears to be Cilium, k3s, containerd or the node itself

Three commands worth memorizing

Network

```
kubectl debug -n "$NS" -it "pod/$POD" \
  --container="netshoot-$(date +%s)" \
  --target="$CONTAINER" \
  --image=nicolaka/netshoot \
  --profile=netadmin \
  -- zsh
```

Java

```
kubectl debug -n "$NS" -it "pod/$POD" \
  --container="java-debug-$(date +%s)" \
  --target="$CONTAINER" \
  --image=eclipse-temurin:21-jdk-jammy \
  --profile=general \
  -- bash
```

Node

```
kubectl debug -it "node/$NODE" \
  --image=nicolaka/netshoot \
  --profile=sysadmin \
  -- zsh
```

Production recommendations

- Mirror approved debug images into your internal registry.
- Pin exact versions or immutable digests instead of floating tags.

- Define RBAC for who may create ephemeral containers and privileged node-debug sessions.
- Use profile=general by default; escalate to netadmin or sysadmin only when required.
- Delete temporary debug Pods after the incident.
- Treat heap dumps, JFR files, packet captures and profiler output as potentially sensitive incident artifacts.
- Keep Alloy/Pyroscope as persistent observability; keep the other images primarily on-demand.

Compact reference

Need	Image / approach
Network	nicolaka/netshoot + kubectl debug --profile=netadmin
JVM	eclipse-temurin:21-jdk-jammy + kubectl debug --target=
Spring internals	hengyunabc/arthas
Database	tazhate/k8s-debug-container-db as a separate Pod
PVC / storage	tazhate/k8s-debug-container-storage with the target PVC mounted
Linux / perf	tazhate/k8s-debug-container-perf; sysadmin only when needed
Whole-node profile	intel/gprofiler
Historical Java profiles	grafana/alloy + Pyroscope